

PrintHub

MANUAL DE USUARIO EXTENSO Y DOCUMENTACIÓN TÉCNICA INTEGRAL

ENTORNO DE PRODUCCIÓN Y DESARROLLO v1.0

Proyecto:	Plataforma E-Commerce PrintHub (Backend & Frontend)
Entorno Cloud:	Amazon Web Services (AWS) con Docker & CI/CD
Módulo Académico:	DDAW + NUV (Fines Educativos)
Fecha de Generación:	1 de Junio de 2026
Estado General:	✓ TOTALMENTE OPERATIVO

1. Descripción General del Sistema

El ecosistema digital de **PrintHub** es una plataforma integral de comercio electrónico (e-commerce) diseñada bajo criterios de alta disponibilidad, seguridad y escalabilidad horizontal. Arquitectónicamente, el sistema se encuentra completamente desacoplado en dos componentes nucleares: un **Backend API RESTful** desarrollado sobre el framework de última generación Laravel 11, y una interfaz de usuario o **Frontend SPA (Single Page Application)** construida con Vue 3 y Vite.

Esta documentación constituye el manual técnico y de usuario unificado para la administración, despliegue y desarrollo del proyecto. El sistema centraliza flujos comerciales críticos, automatizando las siguientes dimensiones del negocio:

- **Gestión de Usuarios y Roles:** Control de accesos segregado para administradores del sistema y clientes finales.
- **Mecanismo de Autenticación:** Implementación de tokens ligeros y seguros a través de Laravel Sanctum para un intercambio sin estado.
- **Catálogo Dinámico de Productos:** Administración CRUD integral de artículos disponibles para personalización e impresión.
- **Procesamiento de Pedidos:** Motor transaccional para la creación, seguimiento y persistencia de órdenes de compra.
- **Carrito de Compra Inteligente:** Gestión fluida de ítems temporales vinculados al estado de sesión del usuario.
- **Módulo Administrativo:** Panel de control centralizado con permisos elevados para la explotación de métricas e inventarios.

2. Configuración de Red y Arquitectura DNS

El enrutamiento de tráfico hacia la plataforma PrintHub se efectúa mediante una zona DNS delegada de alta fiabilidad, garantizando que tanto las interacciones de los usuarios en la interfaz web como las llamadas estructuradas a la API se canalicen al destino de infraestructura cloud correcto.

2.1. Dominios y Subdominios Configurados

El direccionamiento está segmentado para aislar el tráfico de presentación del lógico-transaccional:

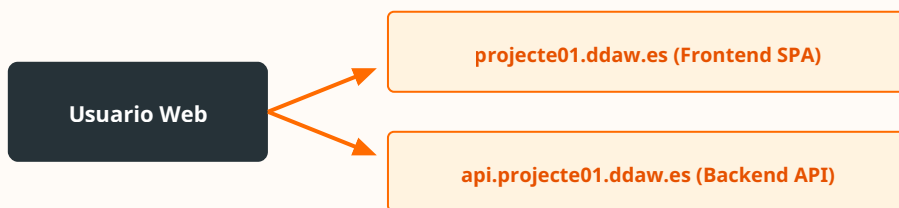
Servicio	Dominio / Subdominio Oficial	Propósito Técnico
Frontend SPA	projecte01.ddaw.es	Interfaz pública del usuario, sirve archivos estáticos optimizados.
Backend API	api.projecte01.ddaw.es	Punto de entrada RESTful para el procesamiento de datos y consultas lógicas.

2.2. Registros Técnicos de la Zona DNS

Para mapear los nombres de dominio con los recursos físicos dentro de Amazon Web Services (AWS), se ha estructurado la siguiente tabla de registros de zona:

Tipo de Registro	Nombre del Host	Destino / Valor	Descripción
A	projecte01.ddaw.es	IP Pública AWS (ALB / Ingress)	Apunta el dominio raíz directamente al balanceador de carga.
CNAME	api.projecte01.ddaw.es	Backend EC2 DNS o ALB Endpoint	Enruta las llamadas API hacia la capa lógica del backend.
CNAME	www.projecte01.ddaw.es	Frontend Host Alias	Redirección canónica estándar para accesos con el prefijo web tradicional.

FIGURA 2.1: MAPEO Y ARQUITECTURA DE LA ZONA DNS DELEGADA



3. Arquitectura de Infraestructura Cloud en AWS

La infraestructura de PrintHub se aloja en Amazon Web Services bajo una topología de Nube Privada Virtual (VPC) propia y dedicada, garantizando el aislamiento perimetral de los datos y permitiendo el crecimiento elástico según la demanda.

3.1. Especificaciones de la VPC (Virtual Private Cloud)

El direccionamiento privado interno sigue una estrategia estricta de subneteo distribuido en múltiples zonas de disponibilidad (Multi-AZ) para evitar puntos únicos de fallo:

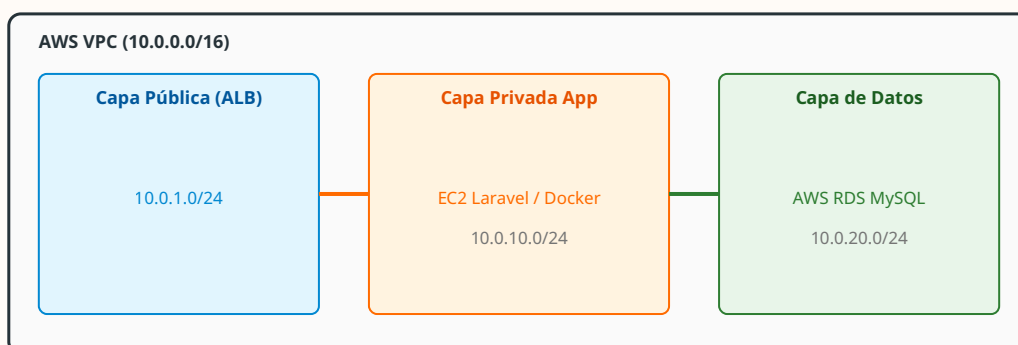
Parámetro de Red	Valor de Configuración	Objetivo Operativo
Bloque CIDR Principal	10.0.0.0/16	Espacio global que proporciona hasta 65,536 direcciones privadas IP.
Zonas de Disponibilidad (AZs)	2 Zonas Autónomas	Soporte para redundancia física cruzada frente a caídas de data centers.
Internet Gateway (IGW)	Configurado y Vinculado	Permite la comunicación bidireccional entre la VPC y la red pública de Internet.
NAT Gateway	Activo en Subred Pública	Permite a las instancias privadas descargar actualizaciones sin exponerse a la red.

3.2. Segmentación Temática de Subredes

La VPC se divide de forma estricta en tres capas lógicas para asegurar el principio de mínimo privilegio de red:

- Subredes Públicas (Ingreso de Tráfico):** Ubicadas en los rangos 10.0.1.0/24 y 10.0.2.0/24 . Alojamos exclusivamente los Application Load Balancers (ALB) orientados a Internet. Son las encargadas de gestionar la terminación de las sesiones criptográficas HTTPS.
- Subredes Privadas de Aplicación (Capa Lógica):** Establecidas en los rangos 10.0.10.0/24 y 10.0.11.0/24 . En ellas residen las instancias EC2 y los contenedores Docker que ejecutan el motor de Laravel 11. No tienen asignadas IPs públicas.
- Subredes Privadas de Datos (Persistencia):** Confinadas en los rangos 10.0.20.0/24 y 10.0.21.0/24 . Es el segmento de red más aislado, reservado única y exclusivamente para los nodos de bases de datos relacionales administradas mediante AWS RDS.

FIGURA 3.1: TOPOLOGÍA DE SUBREDES Y CAPAS DE AISLAMIENTO EN AWS



4. Capa Edge, Políticas de Seguridad y HTTPS

La seguridad perimetral de PrintHub está diseñada bajo el modelo de defensa en profundidad, garantizando que todo el tráfico entrante sea inspeccionado y cifrado de forma obligatoria.

4.1. Punto Único de Entrada: Application Load Balancer (ALB)

No se permite bajo ninguna circunstancia el acceso directo de usuarios de Internet a los servidores que contienen el código fuente del backend. Toda solicitud debe pasar por el ALB, el cual desempeña las siguientes funciones críticas:

- **Redirección Forzada HTTP a HTTPS:** Cualquier petición entrante en el puerto inseguro **80** es reescrita de manera automática hacia el puerto cifrado **443** con un código de estado 301 (Moved Permanently).
- **Terminación Criptográfica SSL/TLS:** El balanceador descifra la petición mediante certificados validados emitidos por la autoridad certificadora **Let's Encrypt**, aliviando la carga de procesamiento computacional a las instancias internas de backend.
- **Algoritmo de Balanceo:** Distribuye equitativamente las peticiones entrantes entre los nodos saludables del pool de Auto Scaling.

4.2. Configuración Estricta de Security Groups (Cortafuegos)

Los Security Groups de AWS actúan como cortafuegos con estado a nivel de instancia, limitando estrictamente los vectores de entrada:

Capa de Infraestructura	Puerto	Protocolo	Origen de Acceso Permitido	Nivel de Seguridad
Load Balancer (ALB)	80 / 443	TCP	0.0.0.0/0 (Cualquier origen público)	Abierto a Internet (Público)
Backend (Instancias EC2)	80 / 8000	TCP	Solo desde el Security Group del ALB	Aislado (Privado)
Base de Datos (RDS)	3306	TCP	Solo desde el Security Group del Backend	Restringido e Inaccesible desde el Exterior

5. Capa de Aplicación: Backend API Laravel 11

La lógica computacional del negocio reside en un desarrollo RESTful optimizado sobre Laravel 11 y PHP 8.2. El diseño del backend es totalmente apátrida (stateless), lo que habilita mecanismos nativos de replicación horizontal y escalado automático basados en el uso de CPU y volumen de peticiones concurrentes.

El sistema utiliza **Laravel Sanctum** para emitir tokens criptográficos de un solo uso que se adjuntan en las cabeceras HTTP (`Authorization: Bearer [token]`) de las peticiones del frontend, garantizando una comunicación ligera y protegida contra ataques de falsificación de peticiones en sitios cruzados (CSRF).

6. Capa de Datos: AWS RDS Administrado

El almacenamiento relacional de PrintHub confía su persistencia a una instancia de motor de base de datos MySQL desplegada en el servicio administrado **AWS RDS**. Sus características de alta resiliencia corporativa comprenden:

- **Despliegue Multi-AZ Activo:** Sincronización en tiempo real de los datos hacia una instancia espejo en una zona secundaria. En caso de desastre físico en la zona principal, AWS conmuta el tráfico de datos automáticamente en segundos sin intervención humana.
- **Backups Automatizados:** Ventana diaria de copias de seguridad con retención histórica y capacidad de restauración en un punto específico en el tiempo (Point-in-Time Restore).
- **Réplicas de Lectura:** Capacidad para añadir nodos esclavos de consulta para descargar de trabajo al nodo maestro transaccional durante picos de tráfico analítico.
- **Acceso Público Denegado:** La instancia carece de una interfaz de red externa, impidiendo ataques de fuerza bruta directos desde el exterior de la VPC.

IMPORTANTE PARA ADMINISTRADORES DE SISTEMA

Cualquier labor de mantenimiento directo sobre la base de datos de producción (como correcciones manuales de datos o inspecciones) requiere obligatoriamente el uso de un túnel SSH (Bastion Host) o la ejecución de comandos desde dentro de los propios contenedores autorizados en la subred de backend.

7. Entorno de Desarrollo Local con Docker: Backend

Con el fin de garantizar la paridad entre los entornos de desarrollo, pruebas y producción, el backend dispone de una arquitectura de contenedores Docker que aísla los componentes de software, evitando conflictos de dependencias locales.

7.1. Estructura de Servicios Dockerizados

El entorno local controlado levanta tres contenedores principales definidos en el manifiesto `docker-compose.yml` :

- `app` : Contenedor con PHP 8.2-FPM y las extensiones necesarias para Laravel.
- `nginx` : Servidor web ligero configurado como proxy inverso local para canalizar las peticiones HTTP hacia el puerto expuesto.
- `mysql` : Instancia de base de datos local que simula el entorno de RDS de AWS.

7.2. Configuración de Variables de Entorno (Archivo `.env`)

Para la correcta interconexión de los contenedores locales, el archivo de configuración debe estructurarse con los siguientes parámetros precisos:

```
DB_CONNECTION=mysql
DB_HOST=mysql_db
DB_PORT=3306
DB_DATABASE=printhub_db
DB_USERNAME=root
DB_PASSWORD=secret
```

7.3. Secuencia de Inicialización del Backend Local

Para desplegar e inicializar el entorno de desarrollo desde una terminal limpia, ejecute la siguiente secuencia secuencial de comandos:

Paso 1: Construcción y arranque de los contenedores en segundo plano

```
$ docker-compose up -d --build
```

Paso 2: Instalación de dependencias lógicas del proyecto con Composer

```
$ docker-compose exec app composer install
```

Paso 3: Generación de la clave criptográfica única de la aplicación

```
$ docker-compose exec app php artisan key:generate
```

Paso 4: Ejecución de las migraciones de base de datos e inyección de datos de prueba

```
$ docker-compose exec app php artisan migrate --seed
```

PERSISTENCIA DE DATOS LOCALES

El contenedor de `mysql` tiene configurado un volumen persistente mapeado en el host de desarrollo. Esto asegura que detener los contenedores mediante `docker-compose down` no causará la pérdida de los registros creados durante las pruebas locales de desarrollo.

8. Pipeline de Integración y Despliegue Continuo (CI/CD)

El flujo de liberación de versiones de PrintHub está totalmente automatizado mediante **GitHub Actions**. Cada cambio integrado en las ramas principales de código desencadena un pipeline estricto de validación y despliegue automatizado, eliminando los errores derivados de operaciones manuales.

8.1. Etapas del Pipeline de Backend

El workflow automatizado se ejecuta sobre contenedores aislados en la infraestructura de GitHub siguiendo el siguiente orden lógico:

1. **Instalación de Dependencias:** Ejecución de un entorno de empaquetado limpio para descargar componentes mediante Composer.
2. **Testing Automático Obligatorio:** Lanzamiento de la suite de pruebas unitarias y de integración de la plataforma mediante el comando interno:

```
$ php artisan test
```

Si alguna de las pruebas definidas falla, el pipeline se aborta inmediatamente y notifica al equipo de desarrollo, prohibiendo el paso a producción.
3. **Despliegue Automatizado por SSH:** Una vez superados los tests, la acción establece una conexión cifrada con el servidor EC2 de producción utilizando el framework de despliegue **Deployer**.

8.2. Gestión de Despliegues Mediante el Usuario `deployer`

Por motivos estrictos de seguridad corporativa, los despliegues no se ejecutan utilizando el usuario administrador del sistema (`root`). Se ha creado un usuario con privilegios acotados denominado `deployer`, cuyas funciones y límites operativos son:

- **Funciones Clave:** Lanzamiento de despliegues automatizados, gestión directa del historial de versiones (releases), ejecución forzada de migraciones en producción y reconfiguración de enlaces simbólicos de servidor.
- **Seguridad del Usuario:** Carece de permisos de administración global del sistema (no `root`). El acceso está restringido de forma exclusiva mediante un par de claves criptográficas SSH asimétricas e intransferibles, estando su uso limitado de forma automatizada a la cuenta de GitHub Actions.

8.3. Ejecución y Estructura Organizativa en Producción

El comando que invoca el despliegue automático desde GitHub Actions es:

```
$ dep deploy production
```

Durante esta fase, se ejecutan de manera desatendida las alteraciones de las tablas en caliente en la base de datos de AWS RDS mediante la bandera de forzado de seguridad de Laravel:


```
$ php artisan migrate --force
```

Deployer gestiona la disponibilidad continua (Zero-Downtime Deployment) estructurando el directorio destino `/var/www/printhub` en los siguientes subdirectorios estratégicos:

Directorio / Enlace	Propósito Operativo en el Servidor de Producción
<code>/current</code>	Enlace simbólico dinámico que apunta a la versión activa y saludable que está sirviendo el tráfico web.
<code>/releases</code>	Historial de carpetas que contienen las últimas versiones desplegadas, permitiendo un Rollback inmediato a la versión anterior si ocurre un fallo imprevisto.
<code>/shared</code>	Almacenamiento común persistente para configuraciones sensibles (archivo <code>.env</code>) y archivos multimedia subidos por los clientes que no cambian entre versiones de código.
<code>/.dep</code>	Metadatos internos, logs técnicos e información de control del sistema de despliegues Deployer.

FIGURA 8.1: FLUJO DE INTEGRACIÓN Y DESPLIEGUE CONTINUO (CI/CD)



9. Capa de Cliente: Frontend Vue 3 SPA

La interfaz con la que interactúan los clientes y administradores de PrintHub es una Single Page Application (SPA) reactiva e instantánea construida sobre el ecosistema moderno de **Vue 3**.

9.1. Stack Tecnológico de Presentación

Para garantizar un rendimiento óptimo en dispositivos móviles y de escritorio, el frontend articula las siguientes librerías de referencia de la industria:

Tecnología Nucleo	Ámbito y Uso Específico en la Aplicación
Vue 3 (Composition API)	Framework estructural para el desarrollo basado en componentes reactivos reutilizables.
Vite	Herramienta de empaquetado y compilación ultra rápida para desarrollo y generación de estáticos de producción.
Pinia	Almacén global de estado para gestionar de forma centralizada datos compartidos (sesión de usuario, carrito).
Vue Router	Mapeador de URLs que emula la navegación web sin recargar la página del navegador (experiencia de app nativa).
Axios	Cliente HTTP encargado de emitir peticiones asíncronas estructuradas hacia el subdominio del backend.
Bootstrap 5	Framework CSS responsable del diseño visual elástico y cumplimiento estricto del enfoque responsive y móvil.

9.2. Dockerización Local del Frontend

El frontend cuenta con su propio entorno de desarrollo en Docker, aislando la necesidad de poseer instalaciones del gestor Node.js localmente en la máquina del desarrollador. El servicio utiliza la variable de entorno `VITE_API_BASE_URL` apuntando a `http://localhost:8000/api` para consumir el backend Docker local.

Para levantar la interfaz local de desarrollo, basta con situarse en la raíz del repositorio de frontend y ejecutar:

```
$ docker-compose up -d --build
```

Una vez levantado el entorno, los ingenieros pueden interactuar y testear la aplicación local en tiempo real a través del puerto asignado por Vite:

```
http://localhost:5173
```

9.3. Proceso de Compilación y Distribución Elástica

Cuando el código pasa la validación de despliegue de GitHub Actions, el pipeline compila los activos de programación a través del compilador interno de producción:

```
$ npm install
$ npm run build
```

Este proceso genera archivos puros, minificados e inmutables de lenguaje **HTML**, **CSS** y **JavaScript** **estáticos** alojados en la carpeta final `/dist`. Dichos archivos son cargados de manera automatizada en buckets de almacenamiento optimizado **Amazon S3** y distribuidos globalmente con latencia ultra-baja a través de la red de entrega de contenido (CDN) **Amazon CloudFront**, dotando al frontend de una capacidad de escalabilidad prácticamente infinita frente a millones de peticiones globales concurrentes.

10. Gobernanza del Código y Normas de Contribución

Para mantener la integridad estructural del software y posibilitar la coexistencia armónica de múltiples programadores trabajando en paralelo, el equipo técnico de PrintHub sigue rigurosas metodologías de control de versiones.

10.1. Estrategia GitFlow de Ramas

El repositorio se gobierna mediante la distribución clásica y segura de ramas de trabajo:

- **main** : Contiene exclusivamente el código estable de producción que está activo de cara a los usuarios. Está totalmente protegida contra commits directos.
- **develop** : Rama central de integración donde confluyen las nuevas características validadas antes de pasar a la ventana de pre-producción.
- **feature/*** : Ramas temporales creadas por los desarrolladores para el diseño de características específicas independientes (ej. `feature/carrito-compras`).

10.2. Protocolo de Revisión y Estilo de Código

Ninguna línea de código alcanza las ramas principales sin someterse a la siguiente auditoría de calidad:

1. **Pull Requests Obligatorios (PR)**: El código nuevo debe solicitar formalmente su integración con la rama central.
2. **Validación Estática Automatizada**: El pipeline revisa que el backend respete las especificaciones internacionales de estilo **PSR-12**.
3. **Revisión por Pares (Peer Review)**: Al menos un Ingeniero Senior del equipo debe inspeccionar de manera visual la lógica antes de otorgar el visto bueno y proceder al Merge definitivo.

11. Cuentas de Acceso de Prueba del Sistema

Para labores de auditoría, control de calidad (QA) y demostraciones operativas en el entorno de producción, se dispone de dos perfiles sembrados de forma fija en el sistema con credenciales estandarizadas:

Rol de Acceso	Correo Electrónico de Autenticación	Contraseña del Sistema	Permisos Asociados
Administrador Global	admin@printhub.com	password	Acceso total a métricas, control de stock, gestión de órdenes de compra de cualquier cliente y parametrización global del e-commerce.
Cliente Estándar	client@printhub.com	password	Permisos orientados a flujo final de compra: navegación, edición del perfil, gestión de su propio carrito y visualización de sus pedidos personales.

12. Cuadro de Mando del Estado del Proyecto

A continuación se detalla la matriz de conformidad técnica y el nivel de madurez operativa de los componentes comprometidos en el alcance de la plataforma PrintHub:

Dimensión del Proyecto	Componente de Infraestructura / Lógica	Estado del Hito
Desarrollo Core	API RESTful (Laravel 11)	✓ COMPLETADO
Desarrollo Core	Vue 3 SPA (Frontend)	✓ COMPLETADO
Virtualización	Contenedores Locales Docker Integrales	✓ COMPLETADO
Infraestructura Cloud	Despliegue Completo en Amazon Web Services (AWS)	✓ COMPLETADO
Seguridad Perimetral	Cifrado SSL/TLS Obligatorio Let's Encrypt (HTTPS)	✓ COMPLETADO
Automatización	Pipeline Integral CI/CD (GitHub Actions)	✓ COMPLETADO
Despliegue Moderno	Zero-Downtime Auto Deploy (Deployer Serverless Flow)	✓ COMPLETADO